

Java程序设计与应用



主讲教师:计算机系翟洁



3. 3. 8 Java常用系统类

本节目录

1

Java API

2

字符串类

3

Math类与Random类

4

Date(日期)类与Calendar (日历) 类



3.8.1 Java API

Java常用包

包 名	主 要 功 能
java.applet	提供了创建applet需要的所有类
java.awt.*	提供了创建用户界面以及绘制和管理图形、图像类
java.beans.*	提供了开发Java Beans需要的所有类
java.io	提供了通过数据流、对象序列以及文件系统实现的系统输入、输出
java.lang.*	Java编程语言的基本类库
java.math.*	提供简明的整数算术及十进制算术的基本方法
java.rmi	提供了与远程方法调用相关的所有类

包 名	主 要 功 能
java.net	提供了用于实现网络通讯应用的所有类
java.security	提供了设计网络安全方案需要的一些类
java.sql	提供了访问和处理来自于Java标准数据源数据的类
java.test	包括以一种独立于自然语言的方式处理文本、日期、数字和消息的类和接口
java.util	包括集合类、时间处理模式、日期时间工具等各类常用工具包
javax.accessibility	定义了用户界面组件之间相互访问的一种机制
javax.naming	为命名服务提供了一系列类和接口
javax.swing	提供了一系列轻量级的用户界面组件，是目前Java用户界面常用的包

3.8.2 字符串类

⌘ 字符串常量：是包含在**双引号**之间的字符序列，例如 `" abc "`。

⌘ 字符串常量可以包含转义字符。

如：`System.out.println(" We are \nfriends! ");`

⌘ **注意：字符串常量与字符常量的区别。**

3.8.2 字符串类

❖ 字符串对象变量:

- 使用之前要声明并进行初始化
- 声明语法格式: **String <字符串变量名>;**
- 利用String类的构造方法创建字符串。

- 例如:

```
s=new String("we are students");
```

- 也可写成:

```
s= "we are students";
```

求指定字符串的长度

```
public class StrLength{  
    public static void main(String[] args){  
        String s1="Hello,Java!";  
        String s2=new String("你好,Java");  
        int len1=s1.length();  
        int len2=s2.length();  
        System.out.println("字符串s1长度为"+len1);  
        System.out.println("字符串s2长度为"+len2);  
    }  
}
```

中文字符也是一个字符长度

字符串s1长度为11
字符串s2长度为7

3.8.2 字符串类

- 🧑 Java程序用字符串池管理字符串，Java程序运行时JVM会在内存创建一个**字符串缓冲池**。
- 🧑 对于Java变量操作，内存主要分为**栈**和**堆**两部分。

字符串缓冲池

栈

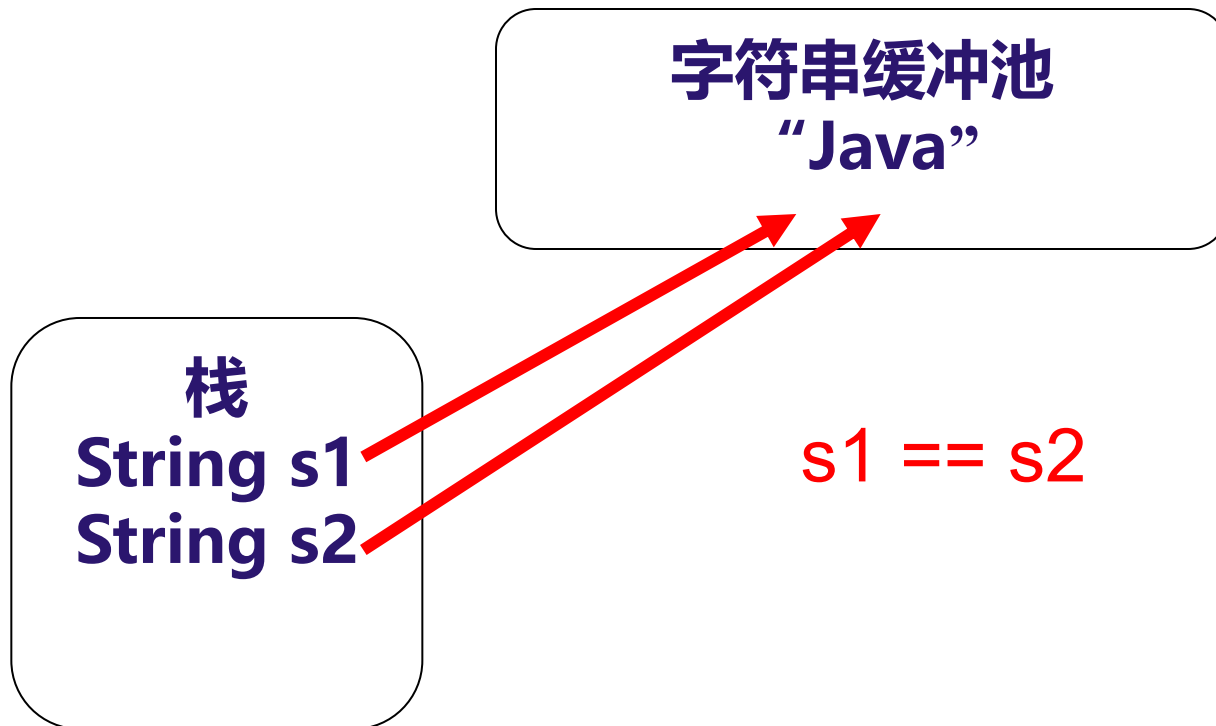
栈用来保存**基本类型**（或者叫内置类型，主要有char、byte、short、int、long、float、double、boolean）和**对象的引用**，数据可以共享。

堆

堆（heap）是用于**存储对象**的**内存空间**，如用new创建的对象。

3.8.2 字符串类

👤 ==: 主要用于判断s1和s2是否引用了同一个对象。



3.8.2 字符串类

- 🧑 Java程序用字符串池管理字符串，Java程序运行时JVM会在内存创建一个字符串缓冲池。
- 🧑 对于Java变量操作，内存主要分为栈和堆两部分。
- 🧑 栈用来保存基本类型（或者叫内置类型，主要有char、byte、short、int、long、float、double、boolean）和对象的引用，数据可以共享。
- 🧑 堆（heap）是用于存储对象的内存空间，如用new创建的对象。
- 🧑 ==：主要用于判断s1和s2是否引用了同一个对象。

【例3.8-2】字符串比较中==运算符的使用。

```
public class StrEqual {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = "Java";  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```



【例3.8-2】字符串比较中==运算符的使用。

```
public class StrEqual {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = "Java";  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```



字符串缓冲池

没有值为
“Java”
的对象

栈

堆



【例3.8-2】字符串比较中==运算符的使用。

```
public class StrEqual {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = "Java";  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```

```
String s1 = "Java";
```

开始有了值为
“Java”的对象

字符串缓冲池
“Java”

栈
String s1

堆



【例3.8-2】字符串比较中==运算符的使用。

```
public class StrEqual {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = "Java";  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```



String s1 = "Java";

String s2 = "Java";

字符串缓冲池
" Java "

栈

String s1
String s2

堆

JVM在字符串缓冲池
中找到" **Java** " —
—**s2**对应的对象，
则**s2**引用**s1**所引用
的对象。



【例3.8-2】字符串比较中==运算符的使用。

```
public class StrEqual {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = "Java";  
        if ( s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```



`s1 == s2?`

字符串缓冲池
“Java”

栈

`String s1`
`String s2`

堆

**s1、s2引用相同
的对象(存储位置
相同)。
所以s1==s2**



【例3.8-2】字符串比较中==运算符的使用。

```
public class StrEqual {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = "Java";  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```

s1==s2



【例3.8-3】字符串对象变量间的赋值运算。

```
public class StrEqual_1 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2;  
        s2=s1;  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```



```
String s1 = "Java";  
String s2;  
s2=s1;
```

字符串缓冲池
"Java"

栈

String s1
String s2

堆

s2指向的仍然是s1指向的"Java"字符串，这里是将s1的引用赋值给s2，s2与s1的引用是相同的。



【例3.8-3】字符串对象变量间的赋值运算。

```
public class StrEqual_1 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2;  
        s2=s1;  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.p  
    }  
}
```

程序运行结果：s1==s2
s2与s1的引用是相同的。

【例3.8-4】在字符串比较中==运算符与equals方法的使用。

```
public class StrEqual_2 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = new String("Java");  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
        if (s1.equals(s2))  
            System.out.println("s1 equals s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```

程序运行结果

s1 != s2

s1 equals s2

【例3.8-4】在字符串比较中==运算符与equals方法的使用。

```
public class StrEqual_2 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
    }  
}
```

}



字符串缓冲池
"Java"

栈
String s1

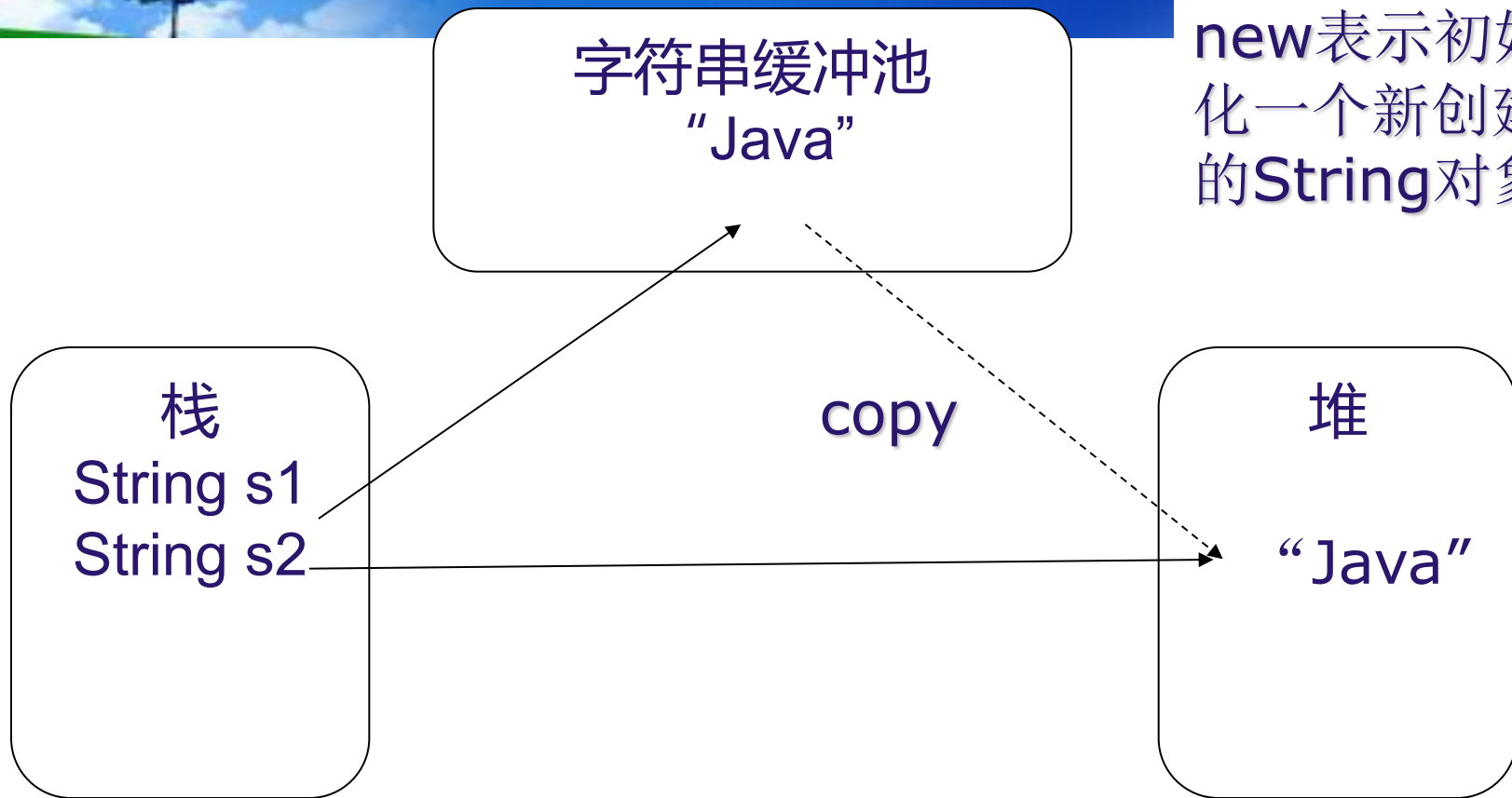
堆

【例3.8-4】在字符串比较中==运算符与equals方法的使用。

```
public class StrEqual_2 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = new String("Java");  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
    }  
}
```

```
String s2 = new String("Java");
```

new表示初始化一个新创建的String对象



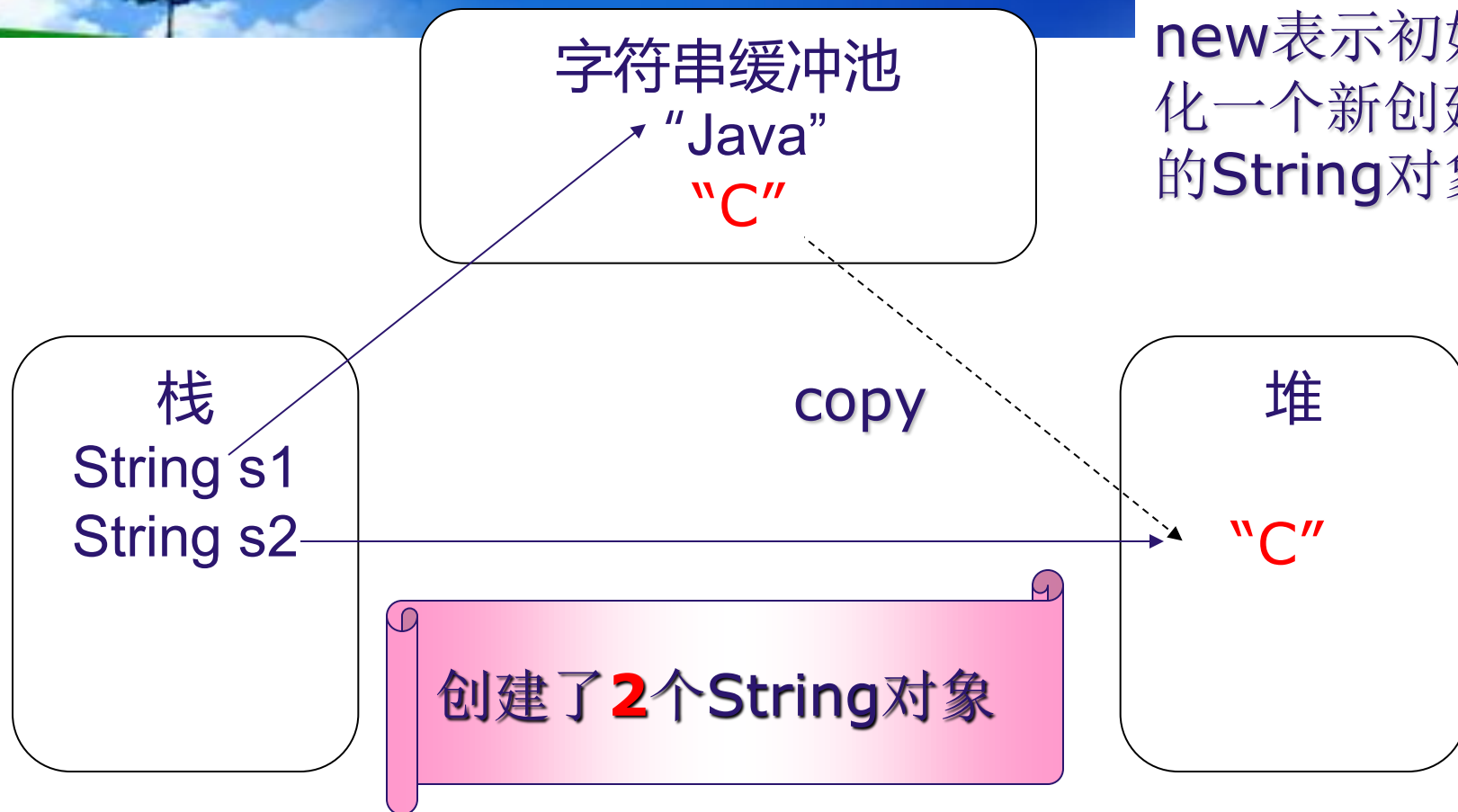
- 1、程序首先在字符串缓冲池中查找是否有“Java”字符串，有则将字符串拷贝到堆中。
- 2、s2指向堆中的“Java”字符串

p161



```
String s2 = new String("C");
```

new表示初始化一个新创建的String对象



- 1、编译期，程序首先在字符串缓冲池中查找是否有“C”字符串，**没有就创建**。然后在运行期则将字符串C拷贝到堆中。
- 2、s2指向堆中的“C”字符串

```
String s2 = new String("Java");
```

new表示初始化一个新创建的String对象

字符串缓冲池
"Java"

栈

String s1
String s2

堆

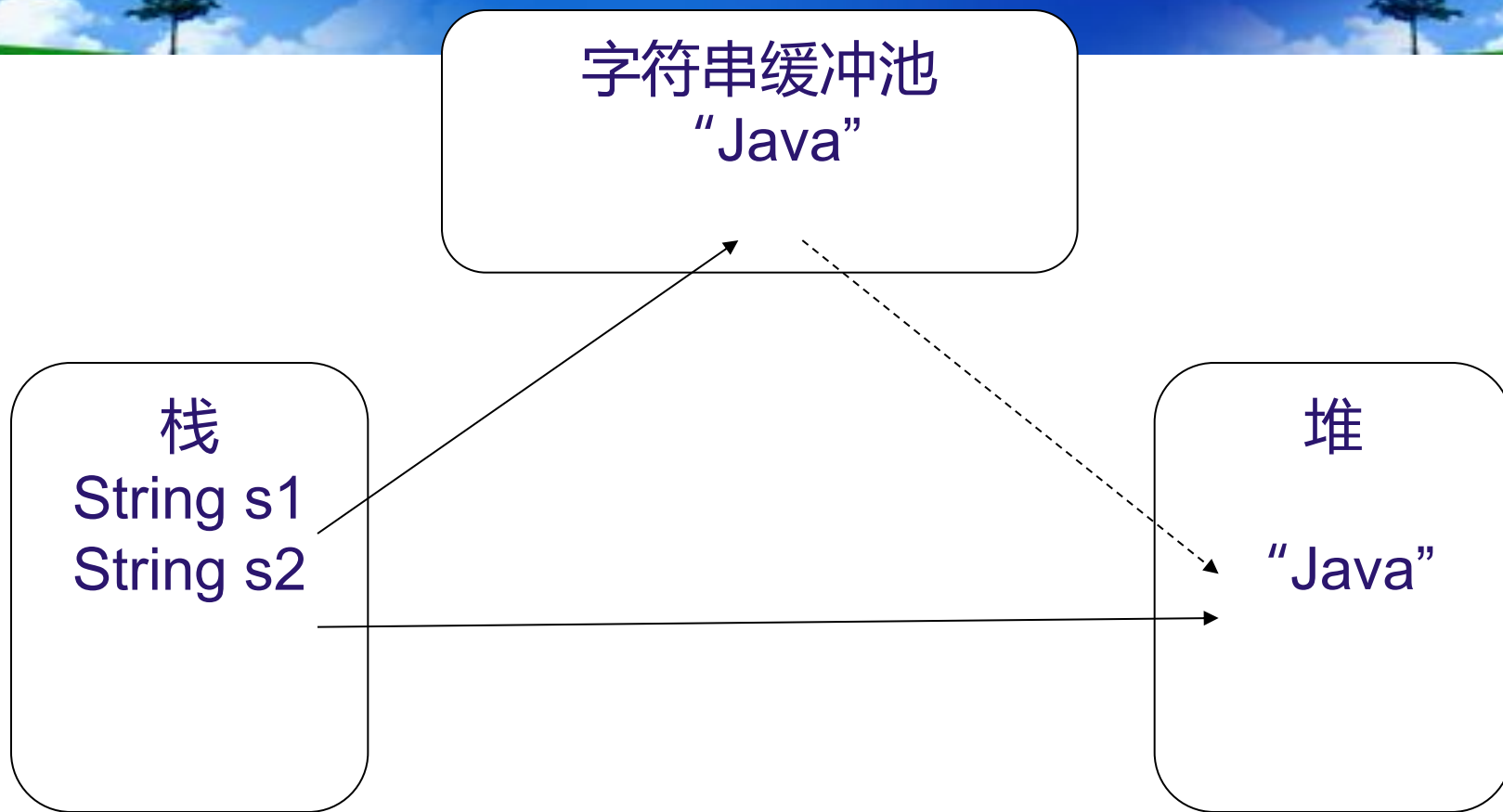
"Java"

因为s1和s2所引用的字符串存储位置不同，
所以s1!=s2

【例3.8-4】在字符串比较中==运算符与equals方法的使用。

```
public class StrEqual_2 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = new String("Java");  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
        if (s1.equals(s2))  
            System.out.println("s1 equals s2");  
        else  
            System.out.println("s1 not equals s2");  
    }  
}
```

```
String s2 = new String("Java");
```



`s1.equals(s2)`是判断**s1**指向的字符串内容是否与**s2**指向的字符串内容相同
所以**s1=s2**

【例3.8-4】在字符串比较中==运算符与equals方法的使用。

```
public class StrEqual_2 {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        String s2 = new String("Java");  
        if (s1 == s2)  
            System.out.println("s1 == s2");  
        else  
            System.out.println("s1 != s2");  
        if (s1.equals(s2))  
            System.out.println("s1 equals s2");  
        else  
            System.out.println("s1 not equals s2");  
    }  
}
```

```
\terminated/ StrEqu  
s1 != s2  
s1 equals s2
```

总结

- ❖ 如果只想判断字符串变量s1、s2的内容是否相同，最好的方式？
- ❖ `s1.equals(s2)`



习题

❖ 以下代码将创建 () 个对象。

❖ String s1="ab";

❖ String s2="ab";

❖ A.0

❖ **B.1**

❖ C.2

❖ D.3

字符串缓冲池
"ab" (仅1个对象)

栈

String s1
String s2

堆



习题

❖ 以下代码将创建 () 个对象。

❖ String s1="ab";

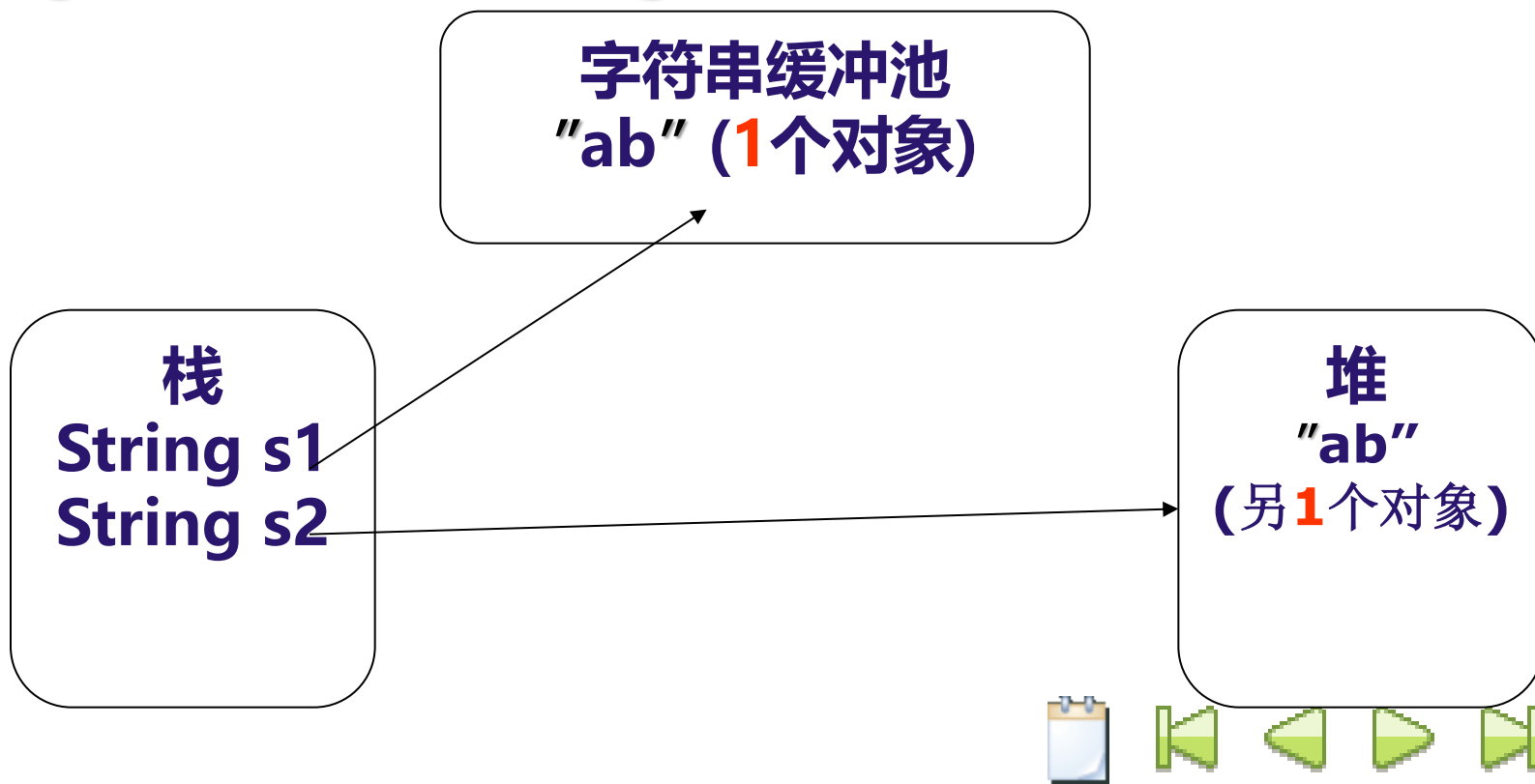
❖ String s2=new String(s1);

❖ A.0

❖ B.1

❖ **C.2**

❖ D.3



❖ 字符串连接符

■ +

- 例如: `s2=s1+","`

■ concat

- 例如: `s3=s2.concat(" Java");`

复制字符串。

`data=s1.toCharArray( );` 将字符串转为数组 `s1=cde`

`System.out.println(" data="+String.valueOf(data));`

`String s2=new String( );` 将数组转为字符串

`String s3=new String( );`

`data=cde`



复制字符串。

```
s2=s1.substring(0);  
System.out.println(" s2="+s2);
```

String substring(int strbegin)

产生一个子串：取父串s[**strbegin,s.lenth-1**] 上字符

s1=cde
0 1 2

s2=cde

复制字符串。

```
s3= s1.substring(1,2);  
System.out.println(" s3="+s3);
```

String substring(int **strbegin**,int **strend**)

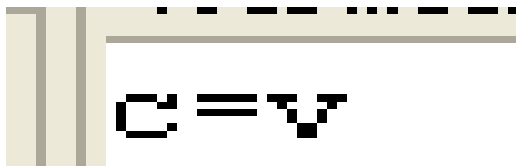
产生一个子串：取父串s[**strbegin**, **strend -1**] 上字符

s1=cde [1,2-1],即[1,1]
 0 1 2

s3=d

【例3.8-8】在字符串中查找字符和子串。

```
public class StrSearch{  
    public static void main(String[] args){  
        String s1="Javav";  
        char c=s1.charAt(2);  
        System.out.println("c="+c);  
    }  
}
```



c=v

J a v a v '
0 1 2 3 4

【例3.8-8】在字符串中查找字符和子串。

```
public class StrSearch{  
    public static void main(String[ ] args){
```

```
        String s1="Javav";
```

```
        int i=s1.indexOf('a');
```

//产生这个字符串中出现给定子字符的**第一个**位置的索引

```
        System.out.println("firstchar="+i);
```

```
        int j=s1.lastIndexOf('a');
```

//产生这个字符串中出现给定子字符的**最后一个**位置的索引

```
        System.out.println("lastchar="+j);
```

```
    }    firstchar=1
```

```
    }    lastchar=3
```

Javav'
0 1 2 3 4

【例3.8-8】在字符串中查找字符和子串。

```
public class StrSearch{  
    public static void main(String[] args){  
        String s1="Javav";  
        i= s1.indexOf("av");  
        System.out.println("firststring="+i);  
        j=s1.lastIndexOf("av");  
        System.out.println("laststring="+j);  
    }  
    firststring=1  
    laststring=3  
}
```

Javav'

0 1 2 3 4

```
String s1="Javav";  
char c=s1.charAt(2);  
System.out.println("c="+c);  
int i=s1.indexOf('a');  
System.out.println("firstchar="+i);  
int j=s1.lastIndexOf('a');  
System.out.println("lastchar="+j);  
i= s1.indexOf("av");  
System.out.println("firststring="+i);  
j=s1.lastIndexOf("av");  
System.out.println("laststring="+j);
```

```
c=v  
firstchar=1  
lastchar=3  
firststring=1  
laststring=3
```

```
String s1="Javav";
```

```
int i1=s1.indexOf('a',2);
```

```
System.out.println("2-fistchar="+i1);
```

int indexOf(String str,int fromIndex)

从给定的索引处开始，产生这个字符串中出现给定子字符的第一个位置的索引。若无此字符，则返回值为-1。

```
2-fistchar=3
```

Javav
0 1 2 3 4

```
String s1="Javav";  
int i1=s1.indexOf('e',2);  
System.out.println("2-fistchar="+i1);
```

int indexOf(String str,int fromIndex)

从给定的索引处开始，产生这个字符串中出现给定子字符的第一个位置的索引。若无此字符，则返回值为-1。

```
\terminated/ >strbear  
2-fistchar=-1
```

Javav'
0 1 2 3 4

练习

❖ 设定一个有大小写的字符串和一个查找字符，使用类String方法indexOf()来判断在该字符串中要查找字符出现的次数。

❖ String s=KB.scan();

❖ char c='e';

```
String s1="Javav";
```

```
int i1=s1.indexOf('a',2);
```

```
System.out.println("2-fistchar="+i1);
```



方法1:

```
System.out.println("请输入一个字符串: ");
String s=KB.scan();
char c='e';
int num=0;
for (int i=0;i<s.length();){
    if (s.indexOf(c, i)>=i){
        num++;
        i=s.indexOf(c, i)+1;
    }
    else
        break;
}
System.out.println("字符串"+s+"有"+num+"个"+c+"字符。");
```



i = 0

s.indexOf('e',0)

= -1 < 0

运行break语句，跳出循环

字符串s

a	a	c
0	1	2
↑		
i		

```
for (int i=0;i<s.length();){  
    if (s.indexOf(c, i)>=i){  
        num++;  
        i=s.indexOf(c, i)+1;  
    }  
    else  
        break;  
}
```



```
i = 0
```

```
s.indexOf('e',0)= 1 > 0
```

```
num=1
```

```
i= s.indexOf('e',0)+1=1+1=2
```

字符串s

a	e	c	d	e
0	1	2	3	4
↑				
i				



i = 2

s.indexOf('e',2) = 4 > 2

num = 2

i = s.indexOf('e',0) + 1 = 4 + 1 = 5

字符串s

a	e	c	d	e
0	1	2	3	4
		↑		
		i		



$i = 5 > s.length() = 4$

循环结束

字符串s

a	e	c	d	e
0	1	2	3	4

↑
i

```
System.out.println("请输入一个字符串: ");
```

```
String s=KB.scan();
```

```
char c='e';
```

```
int num=0;
```

```
for (int i=0;i<s.length();){
```

```
    if (s.indexOf(c, i)>=i){
```

```
        num++;
```

```
        i=s.indexOf(c, i)+1;
```

```
    }
```

```
    else
```

```
        break;
```

```
}
```

```
System.out.println("字符串"+s+"有"+num+"个"+c+"字符。");
```



方法2:

```
System.out.println("请输入一个字符串: ");  
String s=KB.scan();  
char c='e';  
int num=0;  
for (int i=0;i<s.length();i++){  
    if (s.indexOf(c, i)==i){  
        num++;  
    }  
}  
System.out.println("字符串"+s+"有"+num+"个"+c+"字符。");
```

```
i = 0
```

```
s.indexOf('e',0)= 1 != i
```

```
num=0
```

```
i=i+1=0+1=1
```

字符串s

a	e	c	d	e
0	1	2	3	4
↑				
i				



```
i = 1
```

```
s.indexOf('e',1)= 1 = i
```

```
num=1
```

```
i=i+1=1+1=2
```

字符串s

a	e	c	d	e
0	1	2	3	4
	↑			
	i			



i = 2

s.indexOf('e',2)= 4 != i

num=1

i=i+1=2+1=3

字符串s

a	e	c	d	e
0	1	2	3	4
		↑		
		i		



i = 3

s.indexof('e',3)= 4 != i

num=1

i=i+1=3+1=4

字符串s

a	e	c	d	e
0	1	2	3	4
			↑ i	



```
i = 4
```

```
s.indexOf('e',4)= 4 = i
```

```
num=2
```

```
i=i+1=4+1=5 > s.length()
```

字符串s

a	e	c	d	e
0	1	2	3	4
				↑ i



3.8.2 字符串类

⌘ String类型的变量:

- 是一个保存String类对象的**引用**的变量。
- String变量记录String对象在内存中的存放位置。
- 当执行赋值语句时，**原来的引用就被新字符串的引用所覆盖**，原来的字符串被丢弃。

⌘ 例如:

```
String myStr = "my first string"; // 初始化  
myStr = "my second string"; // 改变myStr的值
```



字符串缓冲池
"my first string"

栈
String myStr

堆

String myStr = "my first string"; // 初始化



字符串缓冲池
"my first string"
my second string

栈
String myStr

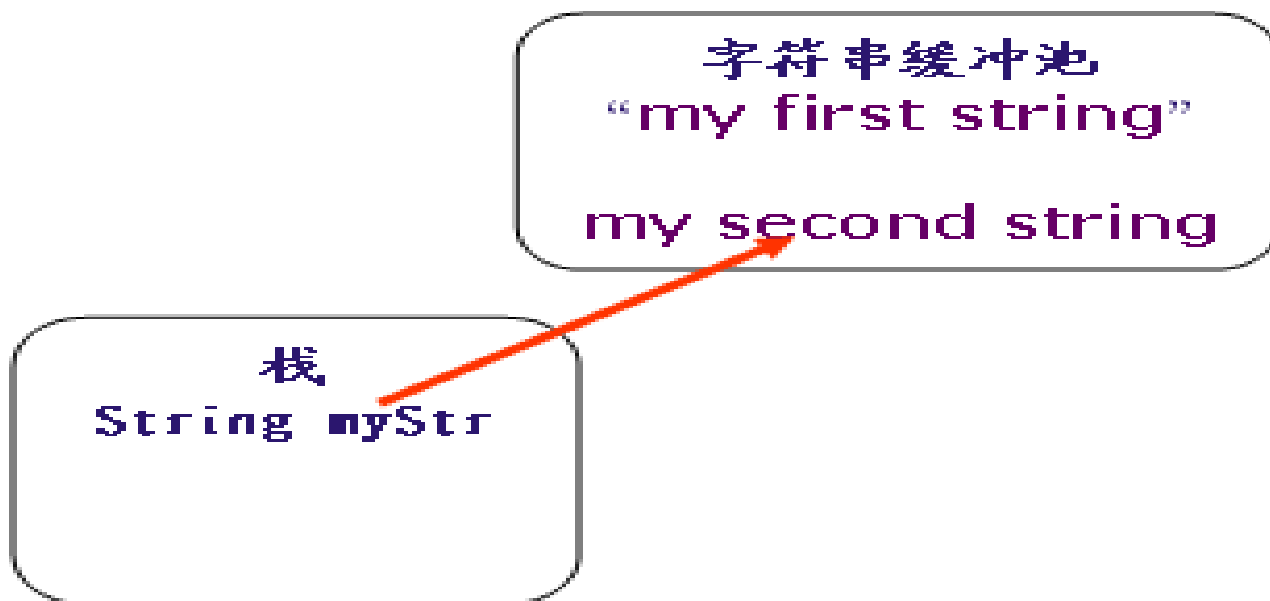
堆

myStr = "my second string"; // 改变myStr的值



3.8.2.2 StringBuffer类

- ❖ String: 对象**不可变**, 可共享。
- ❖ StringBuffer: 支持**可变**的字符串。

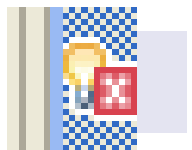


3.8.2.2 StringBuffer类

⌘初始化：必须使用如下方式：

```
StringBuffer str = new StringBuffer("hello");
```

注意：不能像String对象那样只使用字符串做初始值。



```
StringBuffer s1="ab";
```

Type mismatch: cannot convert from String to StringBuffer



3.8.2.2 StringBuffer类

❖ StringBuffer类常用方法:

- **insert(offset, str):** 在位置offset处插入字符串
- **append():** 在其之后追加字符串
- **s1.length():** 返回s1中字符个数。
- **s1.capacity ():** 返回s1的容量, 即内存空间数量, 通常会大于length()。
- **s1.setlength (int newLength):** 改变s1中字符的个数, 如果newLength大于原个数, 则新添的字符都为空 ("") ; 相反, 字符串中的最后几个字符将被删除。

```
StringBuffer s=new StringBuffer();  
s.append("abc");  
s.insert(1,"in");  
System.out.println("s: "+s);  
System.out.println("长度: "+s.length());  
System.out.println("容量: "+s.capacity());
```

因为没有设置初始化字符长度，StringBuffer的容量被初始化为16个字符。

a i n b c

```
s:  a i n b c  
长度:  5  
容量:  16
```



```
s.append("abc01234567890");  
System.out.println("s: "+s);  
System.out.println("长度: "+s.length());  
System.out.println("容量: "+s.capacity());
```

当StringBuffer达到最大容量时，它将自身容量增加到当前值的两倍再加2。

$16 * 2 + 2 = 34$

```
s: ainbcabc01234567890  
长度: 19  
容量: 34
```



```
s: ainbcabc01234567890
```

```
s.setLength(10);  
System.out.println("s: "+s);  
System.out.println("长度: "+s.length());  
System.out.println("容量: "+s.capacity());
```

- **s1.setLength (int newLength)**: 改变**s1**中字符的个数，如果**newLength**大于原个数，则新添的字符都为空（""）；相反，字符串中的最后几个字符将被删除。

```
s: ainbcabc01  
长度: 10  
容量: 34
```

3.8.2.2 StringBuffer类

❖ StringBuffer类常用方法:

- 读取StringBuffer对象中的字符的方法：
charAt()和**getChar()**
- 设置字符及子串的方法：**setCharAt()**和
replace()
- 删除字符及子串的方法：**delete()**及
deleteCharAt()



【例3.8-12】 改变字符串的内容。

```
public class StrChange{
```

```
public static void main(String[] args){
```

```
    StringBuffers1=new StringBuffer(" Hallo,Java! ");
```

```
    s1.setCharAt(1, 'e');
```

```
    System.out.println(s1);
```

H e l l o , J a v a !

```
    s1.replace(1,5, "i");
```

0 1 2 3 4 5 6 7 8 9 10

```
    System.out.println(s1);
```

```
    s1.delete(0,3);
```

```
    System.out.println(s1);
```

He11o, Java!

```
    s1.deleteCharAt(4);
```

```
    System.out.println(s1); } }
```

【例3.8-12】 改变字符串的内容。

```
public class StrChange{
```

```
public static void main(String[] args){
```

```
StringBuffers1=new StringBuffer(" Hallo,Java! ");
```

```
s1.setCharAt(1, 'e');
```

```
System.out.println(s1);
```

H i , J a v a !

```
s1.replace(1,5, "i");
```

0 1 2 3 4 5 6 7 8 9 10

```
System.out.println(s1);
```

```
s1.delete(0,3);
```

[1,5-1]即[1,4]上字符换为i

```
System.out.println(s1);
```

```
s1.deleteCharAt(4);
```

He11o, Java!

Hi, Java!

```
s1.deleteCharAt(4);
```



【例3.8-12】 改变字符串的内容。

```
public class StrChange{
```

```
public static void main(String[] args){
```

```
StringBuffers1=new StringBuffer(" Hallo,Java! ");
```

```
s1.setCharAt(1, 'e');
```

```
System.out.println(s1);
```

J a v a !

```
s1.replace(1,5, "i");
```

0 1 2 3 4 5 6 7

```
System.out.println(s1);
```

```
s1.delete(0,3);
```

[0,3-1]即[0,2]上字符被删除

```
System.out.println(s1);
```

```
s1.deleteCharAt(4);
```

```
He11o,Java!  
Hi,Java!  
Java!
```

```
S
```



【例3.8-12】改变字符串的内容。

```
StringBuffers1=new StringBuffer(" Hallo,Java! ");
```

```
s1.setCharAt(1, 'e');
```

```
System.out.println(s1);
```

```
s1.replace(1,5, "i");
```

```
System.out.println(s1);
```

```
s1.delete(0,3);
```

```
System.out.println(s1);
```

```
s1.deleteCharAt(4);
```

```
System.out.println(s1); } }
```

J a v a

0 1 2 3 4

terminated/ str

Hello, Java!

Hi, Java!

Java!

Java

第4个字符被删除



3.8.2.3 StringTokenizer类

❖ 分析字符串并将字符串分解成可被独立使用的单词，这些单词叫做**语言符号**。

❖ 例如

- 字符串“**We are Students**”，空格作为分隔符，该字符串有三个单词（语言符号）。
- 字符串“**We,are,Student**”，逗号作为分隔符，该字符串有三个单词。

3.8.2.3 StringTokenizer类

❖ 常用构造方法

- (1) StringTokenizer(String s): 为字符串s构造一个分析器. 使用默认的分隔符集合, 即空格符、换行符、回车符、Tab 符等。

```
StringTokenizer fenxi=  
    new StringTokenizer("we are students");  
int number=fenxi.countTokens();  
System.out.println("单词数: "+number);
```

```
单词数: 3
```

3.8.2.3 StringTokenizer类

❖ 常用构造方法

- (2) StringTokenizer(String s,String delim): 为字符串 s 构造一个分析器, 参数 **dilim**中的字符和默认的分隔符集合一起被作为分隔符。

```
StringTokenizer fenxi2=
```

```
    new StringTokenizer("we are ; all;students,!", " , ; ");
```

```
number=fenxi2.countTokens();
```

```
System.out.println("单词数: "+number);
```

单词数: 5

3.8.2.3 StringTokenizer类

❖ 常用构造方法

- (1) StringTokenizer(String s): 为字符串s构造一个分析器。使用**默认**的分隔符集合，即空格符、换行符、回车符、Tab 符等。
- (2) StringTokenizer(String s,String delim): 为字符串 s 构造一个分析器，**参数 delim**中的**字符**和**默认**的分隔符集合一起被作为分隔符。

【例3.8-13】分析给定字符串，分别输出字符串的单词，并统计出单词个数



```
import java.util.*;
public class TokenDemo{
    public static void main(String args[]){
        String s="I am Geng.X.y,she is my girlfriend";
        StringTokenizer fenxi=new
                                StringTokenizer(s," ");
        int number=fenxi.countTokens();
        while(fenxi.hasMoreTokens()){
            String str=fenxi.nextToken();
            System.out.println(str);
            System.out.println("还剩"
                                +fenxi.countTokens()+"个单词");
        }
        System.out.println("s 共有单词 "+number+"个");
    }
}
```



以空格、逗号为分隔符

```
String s="I am Geng.X.y,she is my girlfriend";  
StringTokenizer fenxi=new StringTokenizer(s," ,");  
int number=fenxi.countTokens();  
while(fenxi.hasMoreTokens()){  
    String str=fenxi.nextToken();  
    System.out.println(str);  
    System.out.println("还剩"+fenxi.countTokens()+"个单词");  
}  
System.out.println("s 共有单词 "+number+"个");
```

s 共有单词 7个



String s="I am Geng.X.y,she is my girlfriend";

StringTokenizer fenxi=new StringTokenizer(s," ,");

int number=fenxi.countTokens();

while(fenxi.hasMoreTokens())

只要字符串中还有语言符号，
该方法返回true，否则返回
false

String str=fenxi.nextToken();

System.out.println(str);

System.out.println("还剩"+fenxi.countTokens()+"个单词");

}

System.out.println("s 共有单词 "+number+"个");





```
String s="I am Geng.X.y,she is my girlfriend";
```

```
StringTokenizer fenxi=
```

字符串获得下一个单词:
fenxi对象去掉第一个单词

```
int number=fenxi.countTokens();
```

```
while(fenxi.hasMoreTokens()){
```

am Geng.X.y,she is my girlfriend

```
String str=fenxi.nextToken();
```

```
System.out.println(str);
```

```
System.out.println("还剩"+fenxi.countTokens()+"个单词");
```

```
}
```

```
System.out.println("s 共有单词 " +
```

I
还剩 6 个单词



fenxi对象:



am Geng.X.y,she is my girlfriend

```
int number=fenxi.countTokens();
```

```
while(fenxi.hasMoreTokens()){
```

```
String str=fenxi.nextToken();
```

am

fenxi对象:

```
System.out.println(str);
```

Geng.X.y,she is my girlfriend

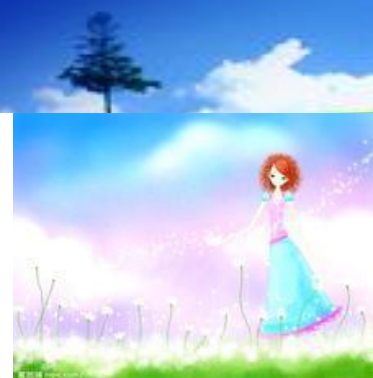
```
System.out.println("还剩"+fenxi.countTokens()+"个单词");
```

```
}
```

```
System.out.println("s 共有单 还剩5个单词
```

fenxi对象:

girlfriend



```
int number=fenxi.countTokens();
```

```
while(fenxi.hasMoreTokens()){
```

```
String str=fenxi.nextToken();
```

```
System.out.println(str);
```

```
System.out.println("还剩"+fenxi.countTokens()+"个单词");
```

```
}
```

```
System.out.println("s 共有单词 "+number+"个");
```

fenxi对象

还剩0个单词





fenxi对象:



所有单词读完，
循环结束

```
int number=fenxi.countTokens();  
while(fenxi.hasMoreTokens()){  
    String str=fenxi.nextToken();  
    System.out.println(str);  
    System.out.println("还剩"+fenxi.countTokens()+"个单词");  
}  
System.out.println("s 共有单词 "+number+"个");
```

s 共有单词 7个





```
public static void main(String args[]){  
    String s="I am Geng.X.y,she is my girlfriend."  
    StringTokenizer fenxi=new StringTokenizer(s," ,");  
    int number=fenxi.countTokens();  
    System.out.println(s+"共有单词 "+number+"个");  
    while(fenxi.hasMoreTokens()){  
        String str=fenxi.nextToken();  
        System.out.println(str);  
        System.out.println(s+"还剩"+fenxi.countTokens()+"个单词");  
    }  
    System.out.println(s+"共有单词 "+number+"个");  
}
```





<terminated> tokenDemo (1) [Java Application] C:\Program Files\

I am Geng.X.y,she is my girlfriend.共有单词 7个

I

I am Geng.X.y,she is my girlfriend.还剩6个单词

am

I am Geng.X. fenxi对象一直在变 end.还剩5个单词

Geng.X.y

但字符串s没变

I am Geng.X. end.还剩4个单词

she

I am Geng.X.y,she is my girlfriend.还剩3个单词

is

I am Geng.X.y,she is my girlfriend.还剩2个单词

my

I am Geng.X.y,she is my girlfriend.还剩1个单词

girlfriend.

I am Geng.X.y,she is my girlfriend.还剩0个单词

I am Geng.X.y,she is my girlfriend.共有单词 7个



回顾

❖ 掌握String类、StringBuffer类及StringTokenizer类中常用方法。

下列说法错误的是（ ）

A. 当字符串的内容会不断修改的时候使用StringBuffer比较合适。

B. **String** 类用字符串池管理字符串

C. **StringTokenizer** 类分析一个字符串并将字符串分解成可被独立使用的单词时使用。

D. String 类是原始基本数据类型

基本类型 (内置类型, 主要有char、byte、short、int、long、float、double、boolean)



3.8.3 Math类与Random类

- ❖ **Math类**: 存在于java.lang包, 可用于求一些数学运算。并有两个双精度静态常量E 和PI, 它们的值分别是自然数e (2.3.8182818284590452354) 和圆周率pi (3.1415926535893.89323846) 。
- ❖ **Random类**: 生成随机数。为使Java程序有良好的可移植性, 应用**Random类**。



3.8.3 Math类

❖ 三角函数方法

构造方法	说 明
<code>public static double sin(double a)</code>	三角函数正弦
<code>public static double cos(double a)</code>	三角函数余弦
<code>public static double tan(double a)</code>	三角函数正切
<code>public static double asin(double a)</code>	三角函数反正弦
<code>public static double acos(double a)</code>	三角函数反余弦
<code>public static double atan(double a)</code>	三角函数反正切



3.8.3 Math类

❖ 指数函数

构造方法	说 明
<code>public static double exp(double a)</code>	返回 e^a 的值
<code>public static double log(double a)</code>	返回 $\ln(a)$ 的值
<code>public static double pow (double y,double x)</code>	返回以y为底数, x为指数的幂值
<code>public static double sqrt(double a)</code>	返回a的平方根

3.8.3 Math类

❖ 舍入函数方法

构造方法	说 明
<code>public static int ceil(double a)</code>	返回大于或等于a的最小整数
<code>public static int floor(double a)</code>	返回小于或等于a的最大整数
<code>public static int abs(int a)</code>	返回a的绝对值
<code>public static int max(int a, int b)</code>	返回a和b的最大值
<code>public static int min(int a, int b)</code>	返回a和b的最小值



3.8.3 Math类

❖ 其他数学方法

构造方法	说 明
<code>public static double random()</code>	返回一个伪随机数，其值介于0和1之间
<code>public static double toRadians(double angle)</code>	将角度转换为弧度
<code>public static double toDegrees(double angle)</code>	将弧度转换为角度

3.8.3 Math类

❖ Math类的这些方法直接通过类名Math调用，例如：

`System.out.println(Math.PI);` 3.141592653589793

`System.out.println(Math.sqrt(9));` 3.0

`System.out.println(Math.random());` 0.061703095869295566

`System.out.println(Math.round(7.5));` 8

`System.out.println(Math.round(7.4));` 7 四舍五入

`System.out.println(Math.max(9,4));` 9

`System.out.println(Math.max(9,4,10));` 出错

用max()方法求数组的最大值

```
int t[]= {21,9,10,8,9,18};
```

示例: Math.max(9,4)

```
public class maxtest {  
    int maxv(          ){ //补充形参  
    int i,maxi=t[0]; //补充下列三行代码  
  
}  
  
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    int t1[]= {21,9,10,8,9,18};  
    maxtest m=          ; //补充对象创建  
    int maxj=m.maxv(    ); //补充实参  
    System.out.println("最大数为: "+maxj);  
}}
```



```
public class maxtest {  
    int maxv(int t[]) {  
        int i,maxi=t[0];  
        for (i=1;i<t.length;i++)  
            maxi=Math.max(maxi,t[i]);  
        return maxi;  
    }  
}
```

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    int t1[]= {21,9,10,8,9,18};  
    maxtest m=new maxtest();  
    int maxj=m.maxv(t1);  
    System.out.println("最大数为: "+maxj);  
}}
```



3.8.3.1 Math类

- ❖ 【例3.8-14】 编程实现两个随机的10以内的整数加法运算题目，共10道题目，要求从键盘输入运算结果，最终显示计算正确的题目数。
- ❖ 程序见MathDemo.java
- ❖ Math.random()

```
num1 = (int) (Math.random() * 10);
```

```
int count=0;
```

```
for (int i=1;i<=10;i++) {
```

```
    int num1,num2,sum=0;
```

```
    num1=(int) (Math.random()*10);
```

```
    num2=(int) (Math.random()*10);
```

```
    System.out.println(num1+" "+num2+"=?");
```

得到2个10以内的随机数



```
try{
    sum=Integer.parseInt(KB.scan());
}
catch(Exception e){
    System.out.println(e.toString());
    e.printStackTrace();
}
```

本方法将把消息和栈跟踪信息输出到标准的错误输出流。

5 + 6 = ?

```
System.out.println(e.toString());
```

```
java.lang.NumberFormatException: For input string: ""
```

```
e.printStackTrace();
```

```
java.lang.NumberFormatException: For input string: ""
```

```
at java.lang.NumberFormatException.forInputString(NumberFormatException.java:48)
```

```
at java.lang.Integer.parseInt(Integer.java:468)
```

```
at java.lang.Integer.parseInt(Integer.java:497)
```

```
at math_random.MathDemo.main(MathDemo.java:15)
```

程序员职责： 预防缺陷，追溯问题！

跟踪信息：
main方法中使用
Integer.parseInt，
由于输入项为空，不
符合整数格式要求。





```
if ( (num1+num2) ==sum) {
```

```
    System.out.println("you are right! go on!");
```

```
    count++;
```

```
else
```

```
    System.out.println("I'm sorry to tell you,you are wrong!");
```

```
}
```

```
System.out.println("你做对了"+count+"个题目!");
```



- ❖ KB.scan()返回的是字符串
- ❖ 以不同形式读取该数据
- ❖ 字符串
- ❖ 1、String s= KB.scan();
- ❖ 整型
- ❖ 2、Int i=Integer.parseInt(KB.scan());
- ❖ 浮点型
- ❖ 3、double
d=Double.parseDouble(KB.scan())
- ❖ 字符
- ❖ 4、char c=s.charAt(0)

3.8.3.2 Random类

❖ Random有两个构造方法:

- Random()(使用系统时间为种子数)
- Random(long seed)

❖ Random具有nextBoolean(), nextInt()等方法, Random包含在java.util包中。

❖ 例如:

- `Random r=new Random( );`
- `int i1=r.nextInt( );` // 返回下一个伪随机数, 是此随机数生成器序列中均匀分布的 int 值
- `int i2=r.nextInt(20);` // 产生大于等于0小于20的随机整数
- `double d1=r.nextDouble( );` //产生大于等于0.0小于1.0的随机数
- `double d1=r.nextDouble(20.0);`
- `// 错, 无参方法`

3.8.3.2 Random类

❖ 【例3.8-15】产生十个不大于100的整数。

```
■ import java.util.*;  
■ public class RandomDemo{  
■     public static void main(String[] args){  
■         Random num=new Random();  
■         for(int i=0;i<10;i++){  
■             System.out.print(num.nextInt(100)+"\t");  
■         }  
■     }  
■ }
```



3.8.3.2 Random类

❖ 【例3.8-15】产生十个不大于100的整数。

```
■ import java.util.*;  
■ public class RandomDemo{  
■ public static void main(String[] args){  
❖     for (int i=0 ; i<10 ; i++)  
❖         System.out.print (new  
❖         Random () .nextInt (100) + "\t" ) ;  
■     }  
■ }
```



3.8.4 Date类与Calendar类

❖ Java标准的日期部分:

- Calendar(日历)
- Date(日期)
- DateFormat(日期格式)

❖ 主要位于java.util包中。

3.8.4.1 Date类

❖ 类 Date :

- 表示特定的瞬间，精确到毫秒。
- 允许把日期解释为年、月、日、小时、分钟和秒值。它也允许格式化和解析日期字符串。
- 从 JDK 1.1 开始，应使用 Calendar 类实现日期和时间字段之间转换，使用 DateFormat 类来格式化和解析日期字符串。

3.8.4.1 Date类

- ❖ 1.直接创建Date数据类型的**实例**变量
- ❖ 创建Date类型的变量date并直接将其输出, 程序运行过程中直接将当前时间打印出来的语句如下:
 - `Date date = new Date();`
 - `System.out.println("当前日期为:" + date);`
- ❖ 当前日期为:Wed Apr 3 21:13:44 CST 2024

3.8.4.1 Date类

❖ 2.将直接实例化的Date数据以**字符串**方式输出

❖ 主要语句如下:

- `Date date=new Date();`
- `String str_date = date.toString();`
- `System.out.println("字符串日期为:" + str_date);`

❖ 字符串日期为:Wed Apr 3 21:23:09 CST 2024

3.8.4.1 Date类

❖ 3.采用DateFormat类的方法格式化或过滤所需要的数据参数。getInstance () 获得**默认**的DateFormat

- Date date=new Date();
- String str_date_1 =
DateFormat.getInstance().format(date);
- System.out.println("格式化日期为:" + str_date_1);
- 格式化日期为:24-4-03 下午10:01

4.采用SimpleDateFormat类的方法格式化或过滤所需要的数据参数，定制日期的格式。

```
import java.text.*;
```

```
SimpleDateFormat time =
```

```
new SimpleDateFormat("yyyy年MM月dd日  
HH:mm:ss");
```

```
System.out.println("日期时间为:"+  
time.format(date));
```

日期时间为:2024年04月03日22:04:08



3.8.4.1 Date类

- ❖ 5.通过getTime()方法获取当前日期的时间
- ❖ 通过getTime()方法所得到的Date类型数据是以毫秒来计算的，并且是以**1970年1月1日**为开始时间。在声明数据类型时需要较大存储空间，可以使用long数据类型或者同等存储类型数据。
- ❖ 主要语句如下：
 - `long str_get = date.getTime();`
 - `System.out.println("当前毫秒值为:" + str_get);`
 - 当前毫秒值为:1493042704685

//1直接创建Date数据类型的实例变量

```
Date date = new Date();
```

```
System.out.println("当前日期为:" + date);
```

//2将直接实例化的Date数据以字符串方式输出

```
String str_date = date.toString();
```

```
System.out.println("字符串日期为:" + str_date);
```

//3采用DateFormat类的方法格式化或过滤所需要的数据参数

```
String str_date_1 = DateFormat.getInstance().format(date);
```

```
System.out.println("格式化日期为:" + str_date_1);
```

//4采用SimpleDateFormat类的方法格式化或过滤所需要的数据参数

```
SimpleDateFormat time =
```

```
new SimpleDateFormat("yyyy年MM月dd日HH:mm:ss");
```

```
System.out.println("日期时间为:" + time.format(date));
```

//5通过getTime()方法获取当前日期的时间

```
long str_get = date.getTime();
```

```
System.out.println("当前毫秒值为:" + str_get);
```



当前日期为:Wed Apr 03 22:05:04 CST 2024
字符串日期为: Wed Apr 03 22:05:04 CST
2024
格式化日期为:24-4- 03下午10:05
日期时间为:2024年04月03日22:05:04
当前毫秒值为:1493042704685



3.8.4.1 Date类

- ❖ 利用DateFormat类实现当前日期格式化输出。
- ❖ 见CurrentDate_1.java
- ❖ **getDateTimelInstanse**的每次调用中都传递两个值：
第一个参数是日期风格，第二个参数是时间风格。
DateFormat 类 提 供 的 常 量：
SHORT,MEDIUM,LONG,FULL。

Date date = new Date();

DateFormat shortDateFormat = DateFormat.getDateInstance(
DateFormat.SHORT, DateFormat.SHORT);

24-4-24 下午10:15

DateFormat mediumDateFormat = DateFormat.getDateInstance(
DateFormat.MEDIUM, DateFormat.MEDIUM);

2024-4-24 22:15:49

DateFormat longDateFormat = DateFormat.getDateInstance(
DateFormat.LONG, DateFormat.LONG);

2024年4月24日 下午10时15分49秒

DateFormat fullDateFormat = DateFormat.getDateInstance(
DateFormat.FULL, DateFormat.FULL);

2024年4月24日 星期一 下午10时15分49秒 CST



```
SimpleDateFormat format1 =
```

```
    new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
```

```
SimpleDateFormat format2 =
```

```
    new SimpleDateFormat("yyyy-MM-dd HH:mm");
```

```
SimpleDateFormat format3 = new SimpleDateFormat("yyyy年MM月dd日");
```

```
SimpleDateFormat format4 = new SimpleDateFormat("yyyy/MM/dd");
```

```
SimpleDateFormat format5 = new SimpleDateFormat("yyyy-MM-dd");
```

```
SimpleDateFormat format6 = new SimpleDateFormat("yyyy-MM");
```

```
SimpleDateFormat format7 = new SimpleDateFormat("yyyy");
```

2024-04-24 22:23:03

2024-04-24 22:23

2024年04月24日

2024/04/24

2024-04-24

2024-04

2024



3.8.5.2 Calendar类

- ❖ Calendar 类是一个**抽象类**，它为特定瞬间与一组诸如 YEAR、MONTH、DAY OF MONTH、HOUR 等**日历字段**之间的转换提供了一些方法，并为操作日历字段（例如获得下星期的日期）提供了一些方法。
- ❖ 特定瞬间可用毫秒值来表示，它是距历元（即格林威治标准时间 1970 年 1 月 1 日的 00:00:00.000）的偏移量。



3.8.5.2 Calendar类

❖ Calendar类对象的创建:

- 不能用new创建实例对象（抽象类），
- 利用 getInstance 方法返回一个 Calendar 对象，其日历字段已由当前日期和时间初始化，
- 例如：

```
Calendar rightNow = Calendar.getInstance();
```

Calendar类

- ❖ 利用Calendar类实现2012年5月的日历显示。
- ❖ 见CaleDome.java

```
System.out.println("2012年5月日历");  
System.out.println(" 日 一 二 三 四 五 六");  
Calendar rili=Calendar.getInstance();  
rili.set(2012,4,1); //将日历翻到2012年5月1日
```

日 一 二 三 四 五 六

** ** 1 2 3 4 5

❖ 【例3.8-18】 利用Calendar类实现2012年5月的日历显示。

❖ 见CaleDome.java

5月1日是星期的第3天
 $\text{xingqi} = 3 - 1 = 2$

```
int xingqi=rili.get(Calendar.DAY_OF_WEEK)-1;  
String a[]=new String[xingqi+31]; //2012年5月的天数31  
for(int i=0;i<xingqi;i++){  
    a[i]="**";  
}
```

日	一	二	三	四	五	六
**	**	1	2	3	4	5



❖ 【例3.8-18】利用Calendar类实现2012年5月的日历显示。见CaleDome.java

```
for(int i=xingqi, n=1; i<xingqi+31; i++) {  
    if (n<=9)  
        a[i]=String.valueOf(n) + " ";  
    else  
        a[i]=String.valueOf(n) ;  
    n++;  
}
```

[1-9]+ " "
等同于两位数
所占位置

因为a[i]是字符串型，
而n是整型，所以将
整型转换为字符串型

日	一	二	三	四	五	六
**	**	1	2	3	4	5
6	7	8	9	10	11	12

❖ 【例3.8-18】利用Calendar类实现2012年5月的日历显示。

❖ **CalendarDemo.java**

```
for(int i=0;i<a.length;i++){  
    if(i%7==0){  
        System.out.println("");  
    } //先换行，再输出结果  
    System.out.print(" "+a[i]);  
}
```

terminated/CalendarDemo Java Appl
2012年5月日历
日 一 二 三 四 五 六

**	**	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		



❖ 如果输入2023年9月，程序哪些地方需要修改？

import java.util.Calendar;

如果输入2023年9月，程序哪些地方需要修改？

```
public class CaleDemo{
```

```
public static void main(String args[]){
```

```
    System.out.println("2022年2月日历");
```

```
    System.out.println(" 日 一 二 三 四 五 六");
```

```
        Calendar rili=Calendar.getInstance();
```

```
    rili.set(2022,1,1); //将日历翻到2022年2月1日
```

```
    int xingqi=rili.get(Calendar.DAY_OF_WEEK)-1;
```

```
    String a[]=new String[xingqi+28]; //2022年2月的天数28
```

```
    for(int i=0;i<xingqi;i++){
```

```
        a[i]="**";
```

```
    }
```

```
    for(int i=xingqi,n=1;i<xingqi+28;i++){
```

```
        if(n<=9)
```

```
            a[i]=String.valueOf(n)+" ";
```

```
        else
```

```
            a[i]=String.valueOf(n) ;
```

```
            n++;
```

```
    }
```

```
    for(int i=0;i<a.length;i++){
```

```
        if(i%7==0){
```

```
            System.out.println("");
```

```
        } //先换行，再输出结果
```

```
        System.out.print(" "+a[i]);    }
```

2022年2月日历

日 一 二 三 四 五 六

** ** 1 2 3 4 5

6 7 8 9 10 11 12

13 14 15 16 17 18 19

20 21 22 23 24 25 26

27 28



- ❖ 如果输入2023年9月，程序哪些地方需要修改？
- ❖ `rili.set(2023,8,1);` //将日历翻到2023年9月1日
- ❖ `String a[]=new String[xingqi+30];`
//2023年9月的天数30
- ❖ `int i=xingqi,n=1;i<xingqi+ 30`



作业

❖ 输入年、月（如2018/5），输出相应的日历。

请输入年份、月份（如2012/5）

2018/5

2018年5月日历

日 一 二 三 四 五 六

**	**	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		



本节小结

❖ Java包

- Java的核心包 (Java core package) , 包名以“java”开始
- Java的扩展包 (Java extension package) , 包名以 “javax”开始。

本节小结

❖ String 类

- 不是原始基本数据类型
- 字符串是一个对象
- 用字符串池管理字符串



本节小结

❖ StringBuffer类

- 是线程安全的可变字符序列。
- 包含某种特定的字符序列，通过某些方法调用可以改变该序列的长度和内容。
- 当字符串的内容会不断修改的时候使用StringBuffer比较合适。

❖ StringTokenizer 类

- 分析一个字符串并将字符串分解成可被独立使用的单词时使用。



本节小结

- ❖ 数学运算：用Math类中的方法
- ❖ 产生随机数：用Math.random()方法或用Random类完成。
- ❖ Date类及Calendar类主要用于日期操作。

Java Programing Language

Thank You !

www.ccu.edu.cn